

Complete Cron Expressions & Linux Cron Jobs Guide

Table of Contents

Complete Cron Expressions & Linux Cron Jobs Guide.....	1
1. Introduction to Cron.....	1
2. Cron Expression Structure.....	2
3. Understanding Each Field.....	2
Minute (0–59).....	2
Hour (0–23).....	2
Day of Month.....	3
Month.....	3
Day of Week.....	3
4. Special Characters.....	3
Asterisk (*).....	3
Comma (,).....	3
Hyphen (-).....	3
Slash (/).....	4
5. Common Cron Expressions.....	4
6. Linux Crontab Basics.....	4
7. User vs System Crontab.....	5
User Crontab.....	5
System Crontab.....	5
8. Managing Cron Jobs.....	5
9. Output Redirection.....	5
10. Environment Variables.....	6
11. Special Cron Keywords.....	6
12. Real-World Examples.....	6
13. Cron Best Practices.....	7
14. Troubleshooting Cron Jobs.....	7
15. Interview Questions.....	7
16. Cron Cheat Sheet.....	8
Easy Reading Rule.....	8

1. Introduction to Cron

Cron is a time-based job scheduler in Linux and Unix systems used to automate repetitive tasks.

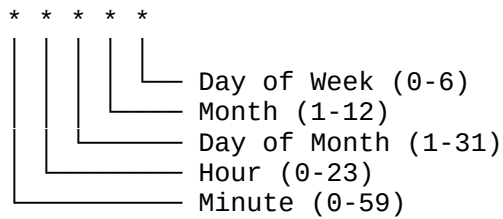
Common uses:

- Database backups
- Log cleanup
- Scheduled reports
- Server monitoring
- Application maintenance

Think of Cron as an alarm clock that automatically runs commands or scripts at scheduled times.

2. Cron Expression Structure

A standard cron expression contains five fields:



Example:

30 14 * * *

Meaning:

- Minute = 30
- Hour = 14 (2 PM)
- Day = Any
- Month = Any
- Weekday = Any

Runs every day at 2:30 PM.

3. Understanding Each Field

Minute (0–59)

15 * * * *

Runs at minute 15 of every hour.

Examples:

0	*	*	*	*	*	Every hour
15	*	*	*	*	*	Every hour at :15
30	*	*	*	*	*	Every hour at :30

Hour (0–23)

0 9 * * *

Runs daily at 9:00 AM.

Examples:

0	0	*	*	*	*	Midnight
---	---	---	---	---	---	----------

0 12 * * * Noon
0 18 * * * 6 PM

Day of Month

0 9 1 * *

Runs on the 1st day of every month.

Month

0 9 * 1 *

Runs every day in January.

Day of Week

0 9 * * 1

Runs every Monday at 9 AM.

0 Sunday
1 Monday
2 Tuesday
3 Wednesday
4 Thursday
5 Friday
6 Saturday

4. Special Characters

Asterisk (*)

Any value.

* * * * *

Every minute.

Comma (,)

Multiple values.

0 9,18 * * *

Runs at 9 AM and 6 PM.

Hyphen (-)

Range.

```
0 9-17 * * *
```

Runs hourly from 9 AM through 5 PM.

Slash (/)

Intervals.

```
*/5 * * * *
```

Runs every 5 minutes.

```
*/10 * * * *
```

Runs every 10 minutes.

```
0 */2 * * *
```

Runs every 2 hours.

5. Common Cron Expressions

Expression	Meaning
* * * * *	Every minute
*/5 * * * *	Every 5 minutes
*/15 * * * *	Every 15 minutes
0 * * * *	Every hour
0 0 * * *	Midnight daily
0 6 * * *	Daily at 6 AM
0 9 * * 1	Monday at 9 AM
0 8 * * 1-5	Weekdays at 8 AM
0 0 1 * *	First day of month
0 0 1 1 *	January 1st

6. Linux Crontab Basics

View cron jobs:

```
crontab -l
```

Edit cron jobs:

```
crontab -e
```

Delete all cron jobs:

```
crontab -r
```

7. User vs System Crontab

User Crontab

```
0 8 * * * /home/user/script.sh
```

No username field.

System Crontab

File:

```
/etc/crontab
```

Example:

```
0 8 * * * root /home/user/script.sh
```

Username field required.

8. Managing Cron Jobs

Edit root cron:

```
sudo crontab -u root -e
```

Edit another user's cron:

```
sudo crontab -u john -e
```

View:

```
sudo crontab -u john -l
```

9. Output Redirection

Save output:

```
* * * * * /home/user/script.sh >> /tmp/output.log
```

Save errors:

```
* * * * * /home/user/script.sh 2>> /tmp/error.log
```

Save both:

```
* * * * * /home/user/script.sh >> /tmp/log.txt 2>&1
```

10. Environment Variables

Set PATH manually:

```
PATH=/usr/local/bin:/usr/bin:/bin
```

Example:

```
PATH=/usr/bin:/bin
```

```
0 8 * * * python3 app.py
```

11. Special Cron Keywords

```
@reboot  
@yearly  
@annually  
@monthly  
@weekly  
@daily  
@midnight  
@hourly
```

Examples:

```
@reboot /home/user/start.sh
```

Runs after system reboot.

```
@daily /home/user/script.sh
```

Runs once per day.

12. Real-World Examples

Database backup:

```
0 1 * * * /usr/local/bin/db_backup.sh
```

Delete temp files:

```
0 3 * * 0 rm -rf /tmp/*
```

Health check:

```
*/5 * * * * /opt/scripts/healthcheck.sh
```

Weekly report:

```
0 7 * * 1 /opt/scripts/report.sh
```

Restart service:

```
0 2 * * * systemctl restart nginx
```

13. Cron Best Practices

1. Use full command paths.
2. Redirect logs.
3. Test commands manually first.
4. Use scripts instead of very long cron lines.
5. Monitor cron logs.
6. Document schedules clearly.
7. Avoid overlapping jobs.

Example:

```
0 8 * * * /usr/bin/python3 /opt/app/report.py
```

14. Troubleshooting Cron Jobs

Check cron service:

```
systemctl status cron
```

or

```
systemctl status crond
```

View logs:

```
grep CRON /var/log/syslog
```

Common issues:

- Wrong permissions
 - Missing execute bit
 - Missing PATH
 - Incorrect script location
 - Wrong cron syntax
-

15. Interview Questions

Q: Difference between * and */5?

A:

* Every value

`*/5` Every 5 values

Q: Difference between 1,3,5 and 1-5?

A:

1,3,5 Specific values

1-5 Continuous range

Q: Run every weekday?

`0 9 * * 1-5`

Q: Run after reboot?

`@reboot`

16. Cron Cheat Sheet

Pattern	Meaning
<code>* * * * *</code>	Every minute
<code>*/5 * * * *</code>	Every 5 minutes
<code>*/10 * * * *</code>	Every 10 minutes
<code>0 * * * *</code>	Every hour
<code>0 0 * * *</code>	Midnight
<code>0 12 * * *</code>	Noon
<code>0 8 * * 1-5</code>	Weekdays 8 AM
<code>0 0 1 * *</code>	Monthly
<code>0 0 1 1 *</code>	Yearly
<code>@hourly</code>	Hourly
<code>@daily</code>	Daily
<code>@weekly</code>	Weekly
<code>@monthly</code>	Monthly
<code>@reboot</code>	On reboot

Easy Reading Rule

Read cron expressions left to right.

Example:

`*/10 9-17 * * 1-5`

Translation:

Run every 10 minutes between 9 AM and 5 PM, Monday through Friday.

Once you can convert cron expressions into plain English, cron becomes easy to understand and troubleshoot.